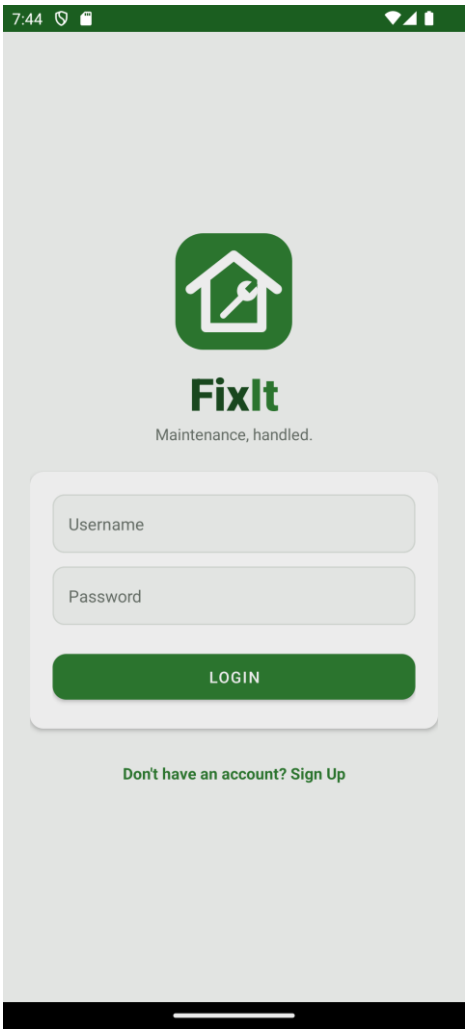
Person 1: Login

FixIt, Landlord–Tenant Maintenance App. Complete work for the Login screen.



The Login screen on the emulator (activity_login.xml, portrait)

What it does

- Shows the FixIt logo, wordmark and tagline above a white card with the username and password fields and the Login button.
- Empty field: toast “Please fill in all fields”.
- Wrong credentials: toast “Username or password is incorrect”.
- Success: saves the user id and role to SharedPreferences, then opens Tenant Home or Landlord Home depending on the role.
- “Don't have an account? Sign Up” opens the Sign Up screen.
- On first start with an empty database it creates the demo accounts and requests (see below).
- User entity with user_id, name, username (unique), password, email, role, unit_number and landlord_id.
- UserDao with insert, login, getByUsername, getLandlordByUsername, getById and countUsers.

Screen components

Component	ID	Notes
ImageView: logo	none	ic_fixit_logo vector, 104dp
TextViews: “Fix” + “It”, tagline	none	Two-tone wordmark, “Maintenance, handled.”
EditText: username	etUsername	Shared input style
EditText: password	etPassword	inputType textPassword
Button: Login	btnLogin	Primary button style
TextView: sign up link	tvGoSignup	Opens SignupActivity

Demo accounts

Created automatically when the database is empty.

Role	Username	Password	Unit	Landlord
Landlord	`landlord1`	`landlord123`		
Landlord	`landlord2`	`landlord123`		
Tenant	`tenant1`	`tenant123`	A-101	landlord1
Tenant	`tenant2`	`tenant123`	A-102	landlord1
Tenant	`tenant3`	`tenant123`	B-201	landlord2

For the rest of the team

- Read the logged-in user with `LoginActivity.PREFS_NAME` (“fixit_prefs”), `KEY_USER_ID` and `KEY_ROLE`.
- Roles are `User.ROLE_TENANT` and `User.ROLE_LANDLORD`.
- Passwords are stored as plain text.

LoginActivity.java

Path: `app/src/main/java/com/fixit/app/LoginActivity.java`

```
package com.fixit.app;

import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class LoginActivity extends AppCompatActivity {

    public static final String PREFS_NAME = "fixit_prefs";
    public static final String KEY_USER_ID = "user_id";
    public static final String KEY_ROLE = "role";

    private final ExecutorService executor = Executors.newSingleThreadExecutor();

    private EditText etUsername;
    private EditText etPassword;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        etUsername = findViewById(R.id.etUsername);
        etPassword = findViewById(R.id.etPassword);
        Button btnLogin = findViewById(R.id.btnLogin);
        TextView tvGoSignup = findViewById(R.id.tvGoSignup);

        seedDemoUsers();

        btnLogin.setOnClickListener(v -> attemptLogin());
        tvGoSignup.setOnClickListener(v ->
                startActivity(new Intent(this, SignupActivity.class)));
    }

    // Demo accounts and requests, inserted once when the database is empty.
    private void seedDemoUsers() {
        executor.execute(() -> {
            AppDatabase db = AppDatabase.getInstance(getApplicationContext());
            UserDao users = db.userDao();
            RequestDao requests = db.requestDao();
            if (users.countUsers() > 0) return;

            int landlord1 = addUser(users, "Landlord 1", "landlord1", "landlord123",
                    User.ROLE_LANDLORD, null, null);
            int landlord2 = addUser(users, "Landlord 2", "landlord2", "landlord123",
                    User.ROLE_LANDLORD, null, null);

            int tenant1 = addUser(users, "Tenant 1", "tenant1", "tenant123",
                    User.ROLE_TENANT, "A-101", landlord1);
            int tenant2 = addUser(users, "Tenant 2", "tenant2", "tenant123",
                    User.ROLE_TENANT, "A-102", landlord1);
            int tenant3 = addUser(users, "Tenant 3", "tenant3", "tenant123",
```

```
                    User.ROLE_TENANT, "B-201", landlord2);

            addRequest(requests, tenant1, "Kitchen sink is leaking",
                    "Water drips from the pipe under the kitchen sink every time the tap is on.
The cabinet floor is getting wet.",
                    Request.STATUS_PENDING, 2, null);
            addRequest(requests, tenant1, "Air conditioner not cooling",
                    "The bedroom AC runs but only blows warm air since yesterday evening.",
                    Request.STATUS_DONE, 12, null);
            addRequest(requests, tenant1, "Front door lock is stuck",
                    "The key turns only halfway and the door is hard to lock at night.",
                    Request.STATUS_PENDING, 1, null);

            addRequest(requests, tenant2, "Bathroom light keeps flickering",
                    "The ceiling light in the bathroom flickers and sometimes turns off by
itself.",
                    Request.STATUS_DONE, 8, null);
            addRequest(requests, tenant2, "Repaint the living room walls",
                    "I would like the living room walls repainted in a lighter color.",
                    Request.STATUS_REJECTED, 5, Request.REJECT_REASONS[0]);

            addRequest(requests, tenant3, "Water heater is not working",
                    "There has been no hot water for two days. The heater makes a clicking
sound but does not heat.",
                    Request.STATUS_PENDING, 1, null);
            addRequest(requests, tenant3, "Broken window latch",
                    "The latch on the bedroom window broke off and the window does not close
tightly.",
                    Request.STATUS_PENDING, 3, null);
            addRequest(requests, tenant3, "Wardrobe door came off",
                    "The left door of the built-in wardrobe came off its hinge.",
                    Request.STATUS_REJECTED, 9,
                    Request.REJECT_REASONS[1] + Request.REASON_SEPARATOR +
Request.REJECT_REASONS[3]);
        });
    }

    private int addUser(UserDao dao, String name, String username, String password,
                        String role, String unit, Integer landlordId) {
        User user = new User();
        user.name = name;
        user.username = username;
        user.password = password;
        user.email = username + "@example.com";
        user.role = role;
        user.unitNumber = unit;
        user.landlordId = landlordId;
        return (int) dao.insert(user);
    }

    private void addRequest(RequestDao dao, int tenantId, String title, String description,
                            String status, int daysAgo, String rejectReasons) {
        Request request = new Request();
        request.tenantId = tenantId;
        request.title = title;
        request.description = description;
        request.status = status;
        request.createdAt = System.currentTimeMillis() - daysAgo * 24L * 60L * 60L * 1000L;
        request.rejectReasons = rejectReasons;
        dao.insert(request);
    }

    private void attemptLogin() {
        String username = etUsername.getText().toString().trim();
        String password = etPassword.getText().toString();

        if (username.isEmpty() || password.isEmpty()) {
```

```
            Toast.makeText(this, "Please fill in all fields", Toast.LENGTH_SHORT).show();
            return;
        }

        executor.execute(() -> {
            User user = AppDatabase.getInstance(getApplicationContext())
                    .userDao().login(username, password);
            runOnUiThread(() -> {
                if (user == null) {
                    Toast.makeText(this, "Username or password is incorrect",
                            Toast.LENGTH_SHORT).show();
                } else {
                    onLoginSuccess(user);
                }
            });
        });
    }

    private void onLoginSuccess(User user) {
        getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
                .edit()
                .putInt(KEY_USER_ID, user.userId)
                .putString(KEY_ROLE, user.role)
                .apply();

        Class<?> home = User.ROLE_LANDLORD.equals(user.role)
                ? LandlordHomeActivity.class
                : TenantHomeActivity.class;
        startActivity(new Intent(this, home));
        finish();
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        executor.shutdown();
    }
}
```

activity_login.xml

Path: `app/src/main/res/layout/activity_login.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/fixit_background"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center_vertical"
        android:orientation="vertical"
        android:padding="24dp">

        <ImageView
            android:layout_width="104dp"
            android:layout_height="104dp"
            android:layout_gravity="center_horizontal"
            android:contentDescription="FixIt logo"
            android:src="@drawable/ic_fixit_logo" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:gravity="center"
            android:orientation="horizontal">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:fontFamily="sans-serif-black"
                android:text="Fix"
                android:textColor="@color/fixit_logo_dark"
                android:textSize="40sp" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:fontFamily="sans-serif-black"
                android:text="It"
                android:textColor="@color/fixit_primary"
                android:textSize="40sp" />
        </LinearLayout>

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="24dp"
            android:gravity="center"
            android:text="Maintenance, handled."
            android:textColor="@color/fixit_text_secondary"
            android:textSize="14sp" />

        <androidx.cardview.widget.CardView
            style="@style/Widget.FixIt.Card"
            android:layout_width="match_parent"
            android:layout_height="wrap_content">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
```

```
                android:orientation="vertical"
                android:padding="20dp">

                <EditText
                    android:id="@+id/etUsername"
                    style="@style/Widget.FixIt.Input"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:autofillHints="username"
                    android:hint="Username"
                    android:imeOptions="actionNext"
                    android:inputType="text"
                    android:maxLines="1" />

                <EditText
                    android:id="@+id/etPassword"
                    style="@style/Widget.FixIt.Input"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="12dp"
                    android:autofillHints="password"
                    android:hint="Password"
                    android:imeOptions="actionDone"
                    android:inputType="textPassword"
                    android:maxLines="1" />

                <Button
                    android:id="@+id/btnLogin"
                    style="@style/Widget.FixIt.Button.Primary"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="20dp"
                    android:text="Login" />
            </LinearLayout>
        </androidx.cardview.widget.CardView>

        <TextView
            android:id="@+id/tvGoSignup"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:clickable="true"
            android:focusable="true"
            android:gravity="center"
            android:minHeight="48dp"
            android:text="Don't have an account? Sign Up"
            android:textColor="@color/fixit_primary"
            android:textSize="14sp"
            android:textStyle="bold" />

    </LinearLayout>
</ScrollView>
```

User.java

Path: `app/src/main/java/com/fixit/app/User.java`

```
package com.fixit.app;

import androidx.room.ColumnInfo;
import androidx.room.Entity;
import androidx.room.Index;
import androidx.room.PrimaryKey;

@Entity(tableName = "users", indices = {@Index(value = "username", unique = true)})
public class User {

    public static final String ROLE_TENANT = "tenant";
    public static final String ROLE_LANDLORD = "landlord";

    @PrimaryKey(autoGenerate = true)
    @ColumnInfo(name = "user_id")
    public int userId;

    public String name;

    public String username;

    public String password;

    public String email;

    public String role;

    @ColumnInfo(name = "unit_number")
    public String unitNumber;

    // Set for tenants only: the user_id of their landlord.
    @ColumnInfo(name = "landlord_id")
    public Integer landlordId;
}
```

UserDao.java

Path: `app/src/main/java/com/fixit/app/UserDao.java`

```
package com.fixit.app;

import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;

@Dao
public interface UserDao {

    @Insert
    long insert(User user);

    @Query("SELECT * FROM users WHERE username = :username AND password = :password LIMIT 1")
    User login(String username, String password);

    @Query("SELECT COUNT(*) FROM users")
    int countUsers();

    @Query("SELECT * FROM users WHERE username = :username LIMIT 1")
    User getByUsername(String username);

    @Query("SELECT * FROM users WHERE username = :username AND role = 'landlord' LIMIT 1")
    User getLandlordByUsername(String username);

    @Query("SELECT * FROM users WHERE user_id = :userId LIMIT 1")
    User getById(int userId);
}
```